

PULSO RIOJANO - Guía de Proyecto

PULSO RIOJANO

Guía de proyecto — Build La Rioja Hackathon (1/8)

1. Resumen ejecutivo

PULSO RIOJANO es un agente de arbitraje energético-hídrico: un sistema que cruza el costo de la tarifa eléctrica por hora con el nivel del acuífero disponible, y le dice al productor agrícola riojano **cuándo bombear agua** para gastar menos plata y no agotar el recurso.

No es un dashboard pasivo. Es una herramienta de **decisión accionable**: entrás, ves un número (cuánto podés ahorrar hoy) y una recomendación concreta (a qué hora bombear), y podés mandarte esa alerta al celular.

2. La problemática que resuelve

En La Rioja conviven tres tensiones que casi nunca se resuelven juntas:

- **Agua escasa**: los acuíferos y pozos tienen niveles variables, y bombear sin criterio los agota más rápido.
- **Energía cara y variable**: la tarifa eléctrica no cuesta lo mismo a toda hora.
- **Producción que necesita rentabilidad**: el productor agropecuario (ej. Cerdo de los Llanos) necesita bajar costos operativos para ser viable, y a la vez necesita financiamiento (FOGAPLAR) que depende de mostrar eficiencia.

Hoy estas tres variables se manejan por separado, a ojo, sin datos cruzados. **Nadie le dice al productor: “bombeá a las 14hs, no a las 9hs, y ahorrarás \$X e \$Y litros de agua”**. Ese es el hueco que llena el MVP.

3. Qué construye el MVP (y qué queda explícitamente afuera)

Definir esto por escrito **antes de tocar código** es el primer punto que va a evaluar Tatiana Sorroche. No lo salteen.

Adentro del MVP

1. Un dashboard con 3 métricas grandes: **\$ ahorrados, litros de agua preservados, kWh optimizados**.
2. Una lógica determinista (una función simple, no un agente en bucle) que cruza tarifa horaria + nivel de acuífero y devuelve la ventana horaria recomendada de bombeo.
3. Un llamado “one-shot” a un modelo de IA (una sola llamada, sin loops) que redacta la alerta en lenguaje natural, lista para simular su envío por WhatsApp/Telegram.
4. Datos simulados/mockeados de tarifa y nivel de pozo (no hace falta integración real con sensores).

Afuera del MVP (decisión consciente, decila en el pitch)

- Login o gestión de usuarios.
 - Integración real con la API de WhatsApp Business (se **simula** en pantalla — más abajo se explica por qué).
 - Agentes de IA con razonamiento en bucle o múltiples llamadas encadenadas.
 - Soporte multi-pozo o multi-productor en esta primera versión (un caso bien resuelto > diez casos a medias).
-

4. Por qué esta idea, y no otra: alineación con el jurado

Juez / evaluador	Qué busca	Cómo lo cubre PULSO RIOJANO
Tatiana Sorroche	Un problema, resuelto sin relleno	Una sola función: “cuándo bombear”. Nada de login ni configuración innecesaria
Gonzalo Pozzo (Goncy)	Pragmatismo, velocidad, no reinventar la rueda	Frontend armado con v0 + componentes shadcn/Tailwind, lógica en backend simple, sin infraestructura pesada
Charly Gauto	Cero fallas en la demo en vivo	Lógica determinista (no depende de que la IA “razone” bien en vivo), IA usada solo para redactar texto, con test automatizado como respaldo
Jurado institucional (Aguas Riojanas, Sanatorio, FOGAPLAR)	Impacto real y ROI medible en la provincia	Toca agua + producción + finanzas a la vez, con cifras en pesos en pantalla

5. Objetivos de Desarrollo Sostenible (ODS) alineados

Muchos hackathons y jurados institucionales piden hoy que el proyecto explicita a qué Objetivos de Desarrollo Sostenible (ODS/SDG de la ONU) aporta. No es un trámite: es un criterio real de evaluación en instancias con actores públicos e institucionales, como el jurado de este evento. Nómbrenlo de forma explícita en la presentación (no alcanza con que “se note” implícitamente).

PULSO RIOJANO aporta, de forma directa y sin forzar la relación, a estos ODS:

ODS	Por qué aplica
ODS 6 — Agua limpia y saneamiento	El sistema previene el uso ineficiente/excesivo del recurso hídrico, recomendando cuándo bombear para no agotar el acuífero
ODS 7 — Energía asequible y no contaminante	Optimiza el consumo eléctrico del bombeo, aprovechando las horas de tarifa más baja/eficiente
ODS 12 — Producción y consumo responsables	Promueve un uso más eficiente de recursos (agua + energía) en la cadena productiva agropecuaria
ODS 9 — Industria, innovación e infraestructura	Aplica IA y datos a un proceso productivo tradicional, aportando innovación tecnológica accesible para productores locales

Recomendación de scope: no sumen más de 3-4 ODS. Listar diez ODS “porque suman puntos” genera el efecto contrario — le da munición a Tatiana Sorroche para decir que el proyecto no tiene foco. Elijan los que puedan explicar en una frase cada uno, y listo.

6. Recomendaciones clave para no fallar en vivo

1. **Simular el envío de WhatsApp/Telegram, no integrarlo de verdad.** La API real de WhatsApp Business requiere aprobación y configuración que no van a poder resolver en horas. Muestren la alerta como si se enviara (una notificación mockeada en pantalla) — el jurado evalúa la idea y el dato, no la integración.
2. **Cero agentes en bucle.** La decisión de “cuándo bombear” se calcula con una función común (if/else o una fórmula), no con un agente de IA razonando. Es instantáneo y no se puede alucinar ni caerse.
3. **Un solo llamado a IA, “one-shot”.** Úsenlo solo para convertir el resultado numérico en una frase humana (“Bombeá entre las 14 y las 16hs, vas a ahorrar \$4.200 hoy”). Si esa llamada falla, tengan un texto de respaldo ya escrito (fallback) para que la demo nunca se vea vacía.
4. **Sumen un test con Playwright** sobre el flujo principal (que el dashboard carga, que al tocar “calcular” aparece una recomendación). No hace falta que sea exhaustivo — 15-20 minutos de trabajo, y mencionarlo en el pitch conecta directo con la charla de Charly Gauto, algo que casi ningún otro equipo va a hacer.
5. **Números grandes, tipografía enorme.** En el dashboard, las tres métricas (\$ ahorrados / litros / kWh) tienen que ser lo primero que se ve, en un tamaño que se lea desde el fondo de la sala.
6. **Ensayen el pitch de 3 minutos** con el orden: problema → qué decidieron dejar afuera → cómo lo

construyeron rápido → cómo lo probaron. Ese orden calca las tres charlas del día (Sorroche → Pozzo → Gauto), y el jurado lo va a reconocer.

7. Guía de diseño de frontend (para un equipo que no es fuerte en diseño)

Como el diseño no es el fuerte del equipo, la estrategia es **no inventar un estilo desde cero**: elijan una referencia visual concreta, y pídañle a v0 (o a la herramienta de IA que usen) que siga esa referencia al pie de la letra, en vez de pedir algo genérico como “*hacé un dashboard bonito*”.

Sitios donde buscar referencias reales de UI (no plantillas genéricas)

- [Mobbin](#) — capturas reales de apps y dashboards de producto. Ideal para ver cómo se organiza una pantalla con métricas y alertas.
- [Land-book](#) — galería de landing pages y dashboards curados por calidad.
- [SaaS Landing Page](#) — específico para productos tipo SaaS/dashboard, que es el estilo de PULSO RIOJANO.
- [One Page Love](#) — buen banco de referencias simples y de una sola pantalla, útil dado el tiempo limitado.
- [Dribbble](#) — buscar específicamente “dashboard analytics” o “energy dashboard” para tono visual.
- [shadcn/ui examples](#) — componentes reales (cards, charts, dashboards) ya armados y prolijos, compatibles con lo que genera v0.
- [Tailwind UI](#) — bloques de dashboard listos, útil como referencia de estructura aunque no los copien literal.
- [Godly](#) y [Awwwards](#) — más para inspiración de detalle (tipografía, color), no para copiar estructura completa (son sitios más “creativos” que dashboards funcionales).

Cómo usarlo en la práctica (con poco tiempo)

1. Elijan **una sola referencia** de Mobbin o SaaS Landing Page que muestre un dashboard con 2-3 métricas grandes + una recomendación/alerta. No mezclen cinco estilos distintos.
2. Sáquenle una captura y pásensela a v0 junto con el pedido: “*Hacé un dashboard con este estilo visual: [descripción], con 3 tarjetas de métricas grandes arriba y una sección de alerta/recomendación abajo, en modo oscuro/claro [elijan uno], usando shadcn/ui.*”
3. No agreguen elementos que la referencia no tiene. La disciplina de “copiar bien un estilo simple” da mejor resultado que “improvisar un estilo propio” cuando el diseño no es el fuerte del equipo.

Paleta y tipografía sugerida (por si necesitan un punto de partida)

- Fondo neutro (blanco o gris muy claro) con un solo color de acento fuerte relacionado al agua/energía: un azul-teal (#0E7C86 o similar) para “agua” y un ámbar (#D97706) para “energía/alertas”. No usen más de 2 colores de acento.
- Tipografía: una sola familia sans-serif limpia (Inter, o la que traiga shadcn por defecto) — no mezclen fuentes, y usen tamaños grandes y contrastados para las 3 métricas principales.
- Eviten decoración innecesaria (íconos de más, gradientes múltiples, animaciones). Con el tiempo que tienen, cada elemento visual debe justificar su lugar.

8. Estructura de carpetas del proyecto

Sugerencia simple, pensada para Next.js + v0 (adaptable si usan otro stack):

```
pulso-riojano/
├── app/                                # rutas (Next.js App Router)
│   ├── page.tsx                       # dashboard principal (las 3 métricas + recomendación)
│   └── layout.tsx
├── components/
│   ├── ui/                            # componentes shadcn generados por v0
│   ├── MetricCard.tsx                 # tarjeta de métrica grande ($ / litros / kWh)
│   ├── RecommendationPanel.tsx        # panel con la hora recomendada + botón "enviar alerta"
│   └── AlertPreview.tsx               # simulación visual del mensaje de WhatsApp/Telegram
├── lib/
│   ├── calculo.ts                     # función determinista: tarifa + nivel de pozo -> ventana horaria
│   └── mockData.ts                    # datos simulados de tarifa eléctrica y nivel de acuífero
└── app/api/
```

```

├── generar-alerta/route.ts # única llamada "one-shot" a IA para redactar el mensaje
├── tests/
│   ├── flujo-principal.spec.ts # test de Playwright sobre el flujo core
│   └── README.md # problema, solución, cómo correrlo (útil para el jurado si preguntan)

```

Regla de oro: todo lo que no está en esta lista, no se construye en este hackathon.

9. Plan de trabajo por etapas (según el cronograma real del sábado)

Horario	Etapas	Qué se hace	Quién
10:10 - 10:40	Definición	Escribir en papel/doc: problema, las 3 métricas, la fórmula de decisión (tarifa + nivel → hora recomendada), qué queda afuera	Todo el equipo junto
10:40 - 11:30	Lógica core	Armar calculo.ts con datos mockeados. Probarlo con 3-4 escenarios a mano antes de tocar UI	Quien esté más cómodo con lógica/backend
10:40 - 12:30	Frontend base	Prompt a v0 con la referencia visual elegida, armar dashboard con las 3 tarjetas + panel de recomendación	Resto del equipo, en paralelo
12:30 - 13:00	Integración inicial	Conectar la lógica con el frontend, primera prueba end-to-end	Todo el equipo
13:00 - 15:00	Almuerzo / pausa	—	—
15:00 - 15:40	Llamado IA + simulación de alerta	Armar el endpoint one-shot que redacta el mensaje, con fallback si falla. Armar la vista de "alerta enviada" simulada	1-2 personas
15:40 - 16:00	Pruebas manuales	Correr el flujo completo 5+ veces, revisar que no haya pantallas rotas ni datos raros	Todo el equipo
16:00 - 16:20	Test con Playwright	Un test simple sobre el flujo principal (carga el dashboard, se puede calcular una recomendación)	Quien tenga más soltura técnica
16:20 - 16:30	Pulido final	Revisar números grandes, textos, que no haya errores visibles	Todo el equipo
16:30 - 17:00	Coffee break	Ensayar el pitch mentalmente, últimos ajustes	—
17:00	Evaluación de jurado	Pitch de 3 minutos (ver guion abajo)	—

10. Organización del equipo

Repártanse roles claros desde las 10:10, no en el camino:

- **1 persona en "lógica/datos":** la función determinista y los datos mockeados. Es la parte que no puede fallar.
- **1-2 personas en "frontend/UI":** trabajar con v0 sobre la referencia elegida, armar las pantallas.

- **1 persona en “integración + testing + pitch”:** conecta las piezas, escribe el test de Playwright, y arma el guion de la presentación mientras los demás terminan detalles.

Si son menos de 4, combinen roles pero mantengan siempre a alguien enfocado exclusivamente en que la demo no se rompa (ese rol no se negocia).

11. Checklist final — puntos clave para ganarse al jurado

Antes de las 17:00, repasen esta lista en voz alta como equipo:

- ☐ ¿Podemos explicar el problema en una sola frase, sin enredarnos?
- ☐ ¿Dijimos explícitamente qué decidimos dejar afuera del MVP?
- ☐ ¿Las 3 métricas grandes se ven claramente desde lejos?
- ☐ ¿La demo corrió sin errores al menos 3 veces seguidas?
- ☐ ¿Tenemos un fallback si la llamada a IA falla o tarda?
- ☐ ¿Corrimos el test de Playwright y pasó?
- ☐ ¿Mencionamos en el pitch cómo esto conecta con agua + producción + financiamiento (los tres sponsors)?
- ☐ ¿Nombramos explícitamente los ODS a los que aporta el proyecto (no alcanza con que se note “implícitamente”)?
- ☐ ¿Practicamos el pitch completo al menos una vez con cronómetro?

12. Guion sugerido para el pitch (3 minutos)

1. **(30 seg) El problema:** “En La Rioja, agua, energía y plata se manejan por separado. Nadie cruza esos datos para decirle al productor cuándo conviene bombear.”
2. **(20 seg) La decisión de scope:** “Decidimos no hacer login, no integrar WhatsApp de verdad todavía, y no usar agentes en bucle — priorizamos que la recomendación sea instantánea y confiable.”
3. **(60 seg) Demo en vivo:** mostrar el dashboard, tocar “calcular”, mostrar la recomendación y la alerta simulada. Señalar las 3 métricas grandes.
4. **(30 seg) Cómo lo construimos:** “Usamos v0 para el frontend y nos enfocamos el tiempo en la lógica de decisión, que es determinista — no depende de que la IA razone bien en el momento.”
5. **(20 seg) Cómo lo probamos:** “Automatizamos un test con Playwright sobre el flujo principal para asegurarnos de que esto no se rompa.”
6. **(20 seg) Cierre con impacto y ODS:** repetir las 3 cifras grandes y cerrar mencionando explícitamente los ODS: “Esto no es un dashboard más — es agua, energía y producción sostenible trabajando juntas, alineado a los ODS 6, 7 y 12, para cada productor riojano.”

Documento preparado como guía de trabajo interno para el equipo — Build La Rioja, sábado 1/8.